

Case Study: Petmart Project

Project: [Petmart Project](#)

Type: Pet-products E-commerce Web Application

Technology: Python, Django, SQLite, HTML, CSS, JavaScript, Bootstrap 5

Overview

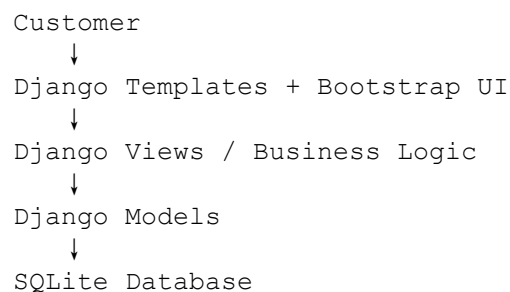
Petmart is an e-commerce web application designed for selling pet food, toys, accessories, and healthcare products online. It supports the complete customer journey: product browsing, cart management, checkout, order placement, feedback, and user profile management.

Technology Stack

Area	Technology Used
Backend	Python, Django
Database	SQLite
Frontend	HTML5, CSS3, JavaScript
UI Framework	Bootstrap 5
Authentication	Django built-in authentication
Icons	Bootstrap Icons
Typography	Google Fonts
Animations	Animate.css
Dynamic updates	Fetch API and Django JSON responses

Application Architecture

The project follows Django's MVT (Model–View–Template) architecture.



- **Models** manage the application data: products, categories, carts, orders, users, feedback, promo codes, and subscriptions.
- **Views** handle business operations such as login, sign-up, cart updates, checkout, order placement, and promo validation.
- **Templates** build the responsive pages seen by customers.

Key Features

Product catalogue

Products are organised through `Category` and `MenuItem` models. Each product can include a name, image, description, price, discounted price, availability, rating, bestseller status, and new-product status.

User authentication

The application uses Django's built-in user authentication system for:

- User registration
- Login and logout
- Customer profile
- Order history
- Extended user profile through `UserProfile`

Shopping cart

The cart supports both guest users and logged-in users.

- Logged-in users have carts linked to their accounts.
- Guest users have carts linked to a session ID.
- Users can add products, update quantities, remove products, and view cart totals.
- Cart operations use AJAX-style JSON responses for a smoother experience.

Checkout and order management

At checkout, the system collects customer details, address information, and payment preference. It calculates:

- Subtotal
- 5% tax
- Delivery fee
- Final total

Orders and individual order items are saved in the database. Customers can later view their order history and order details.

Promo codes and newsletter

The project includes customer-engagement features such as:

- Promo code validation
- Percentage or fixed-value discounts
- Validity periods and usage limits
- Newsletter email subscriptions

Feedback system

Customers can submit feedback with their name, contact details, rating, comments, and an optional photo. Feedback can be moderated through an approval flag.

UI/UX Approach

Petmart uses a custom visual identity designed specifically for a pet-commerce brand.

- Blue, orange, and green colour palette
- Responsive Bootstrap-based layout
- Sticky navigation bar
- Cart badge with item count
- Custom typography using Google Fonts
- Responsive styling for mobile screens
- Hover effects and animations

This gives the project a polished e-commerce appearance instead of a basic CRUD interface.

Core Data Model

```
User → UserProfile
User / Session → Cart → CartItem → MenuItem
Category → MenuItem
User → Order → OrderItem
User / Guest → Feedback
PromoCode
NewsletterSubscriber
```

This structure separates product management, customer information, carts, orders, promotions, and engagement data effectively.

Strengths

- Complete e-commerce workflow from browsing to order confirmation.
- Uses Django's structured backend architecture.
- Supports carts for both guest and authenticated users.
- Includes useful business features such as promo codes, feedback, and newsletter subscriptions.
- Responsive and visually branded user interface.
- Uses Django ORM, built-in authentication, sessions, CSRF protection, and media handling.

Areas for Improvement

Before deploying this project to production, the following improvements are recommended:

- Replace `DEBUG = True`, hard-coded secret keys, and broad `ALLOWED_HOSTS` settings with secure production configuration.
- Use PostgreSQL or MySQL instead of SQLite for scalability.
- Integrate a real payment gateway such as Razorpay or Stripe. Currently, payment methods are stored, but no actual transaction processing is implemented.
- Remove `.env` files and the SQLite database from the public repository if they contain environment-specific or sensitive data.
- Add automated unit and integration tests.
- Add product search, filtering, pagination, and inventory management.
- Add order-confirmation emails, analytics, and stronger admin reporting.
- Store templates, static assets, and applications in a cleaner folder structure for easier maintenance.

Conclusion

Petmart is a well-structured Django-based pet-product e-commerce prototype. It demonstrates a complete customer shopping flow, from product discovery and cart handling to checkout, orders, profiles, feedback, and promotions. Its responsive interface and practical business features make it a strong academic or portfolio project.

With stronger security, payment integration, scalable database support, and automated testing, Petmart can be developed into a production-ready e-commerce platform.